

5교시: Layer/cache/size evidence - 이미지 크기와 rebuild 비용 설명하기



Build Cache와 Layer 최적화

Docker 빌드는 각 명령을 Layer로 만들고, 변경이 없으면 Build Cache를 재사용합니다.

❌ 비효율적인 순서 (Bad)

```
FROM node:18-alpine
COPY . .
RUN npm install
CMD ["node", "app.js"]
```

모든 소스 먼저 복사
↓
소스 변경 시 이후 Layer 모두 다시 빌드

Build 과정과 Cache 상태

Layer	Command	Cache
1	FROM node:18-alpine	cache hit
2	COPY . .	cache miss
3	RUN npm install	cache miss
4	CMD ["node", "app.js"]	cache miss

이후 Layer 모두 재빌드!

⚠️ 소스 파일 하나만 바뀌어도 의존성 설치부터 마지막 Layer까지 다시 빌드됩니다.

✅ 효율적인 순서 (Better)

```
FROM node:18-alpine
COPY package*.json ./ # dependency 먼저
RUN npm install
COPY . . # source 나중
CMD ["node", "app.js"]
```

의존성 파일 먼저 복사
↓
소스 변경 시 소스 복사 이후만 다시 빌드

Build 과정과 Cache 상태

Layer	Command	Cache
1	FROM node:18-alpine	cache hit
2	COPY package*.json ./	cache hit
3	RUN npm install	cache hit
4	COPY . .	cache miss
5	CMD ["node", "app.js"]	cache miss

이 Layer들은 재사용!

이후 Layer만 재빌드!

👍 소스 변경 시, COPY 이후 Layer만 다시 빌드되어 빌드 시간과 리소스 사용을 줄일 수 있습니다.

핵심 포인트

📄 COPY 순서
dependency 먼저
source 나중

Build Cache
변경이 없으면
해당 Layer 재사용

빠른 빌드
불필요한 재빌드를 줄여
빌드 시간 단축

Image size
불필요한 레이어 축적을 줄여
이미지 크기 최적화

비효율적인 순서
불필요한 재빌드로
임시 레이어가 많아져
이미지 크기 증가

효율적인 순서
캐시 재사용으로
불필요한 레이어 축적을 줄여
이미지 크기 최적화

수업 목표

- layer가 Dockerfile instruction의 누적 결과임을 설명한다.
- cache가 이전 layer 재사용임을 설명한다.
- 일반 nginx, Alpine, Trixie 계열 base image로 같은 앱을 빌드해 image size를 비교한다.
- image size가 커질 때 build 속도, pull/push 시간, registry storage, network 비용, 배포 시간에 미치는 영향을 설명한다.

개념 설명

Docker image는 하나의 덩어리처럼 보이지만 내부적으로는 layer가 쌓여 만들어진다. Dockerfile의 instruction은 image history에 흔적을 남긴다. docker history를 보면 base image, file copy, command 같은 build 흔적을 볼 수 있다.

Cache는 이전 build 결과를 재사용하는 기능이다. 변경되지 않은 instruction의 layer는 다시 만들지 않고 재사용할 수 있다. 그래서 Dockerfile 순서와 build context 설계는 rebuild 속도에 영향을 준다. cache는 단순히 빠르게 해주는 기능이 아니라 어떤 변경이 어느 layer부터 영향을 줬는지 보는 evidence다.

Image size도 운영 evidence다. 작은 정적 HTML 앱이라도 base image 선택에 따라 결과 image 크기가 크게 달라진다. Alpine 계열은 보통 작고, Debian 계열 일반/trixie 이미지는 더 크다. 단, 무조건 Alpine이 정답은 아니다. 필요한 패키지, glibc/musl 차이, 디버깅 편의성, 보안 정책, 조직 표준을 함께 봐야 한다.

왜 image size를 줄여야 하는가

영향	image가 커질 때 생기는 문제
Build speed	context와 layer가 커지면 build, cache invalidation, export 시간이 늘어난다.
Push/Pull time	CI runner, registry, Kubernetes node가 image를 주고받는 시간이 늘어난다.
Network cost	registry에서 여러 환경으로 image를 pull하면 egress/traffic 비용이 커질 수 있다.
Storage cost	registry storage, local disk, node image cache 사용량이 늘어난다.
Deploy speed	새 node나 빈 runner가 image를 pull하는 시간이 길어져 rollout이 느려진다.
Security scan	image에 포함된 패키지가 많을수록 scan 시간이 늘고 취약점 표면도 넓어진다.
Debug burden	불필요한 파일이 많으면 무엇이 실제 실행에 필요한지 판단하기 어렵다.

실습 1: 현재 image evidence

```
docker history paperclip-static-site:day3
docker image inspect paperclip-static-site:day3 --format "{{.Id}} {{.Size}}
{{.Architecture}} {{json .RepoTags}}"
```

Expected interpretation:

history에서 COPY index.html, COPY styles.css layer를 확인한다.
inspect의 Size 값은 byte 단위일 수 있다.

실습 2: source 변경 후 cache 확인

```
cd week2/day3/labs/static-site
printf '\n<!-- day3 cache evidence -->\n' >> index.html
docker build -t paperclip-static-site:day3-v2 .
docker images paperclip-static-site
```

Expected interpretation:

변경되지 않은 base/WORKDIR layer는 CACHED로 재사용될 수 있다.

index.html 변경 이후 COPY layer는 다시 실행될 수 있다.

실습 3: base image별 size 비교

이 실습은 같은 앱 파일을 서로 다른 nginx base image 위에 얹어 image size를 비교한다. Docker Hub의 nginx official image에는 stable, stable-alpine, stable-trixie 같은 태그가 있다. 정확한 size는 날짜, platform, Docker cache 상태에 따라 달라질 수 있으므로 학생은 자기 환경의 결과를 기록한다.

비교용 Dockerfile을 먼저 확인한다.

```
cd week2/day3/labs/static-site
sed -n '1,120p' Dockerfile.size-compare
```

Build:

```
docker build -f Dockerfile.size-compare --build-arg BASE_IMAGE=nginx:stable -t
paperclip-static-site:size-default .
docker build -f Dockerfile.size-compare --build-arg BASE_IMAGE=nginx:stable-
alpine -t paperclip-static-site:size-alpine .
docker build -f Dockerfile.size-compare --build-arg BASE_IMAGE=nginx:stable-
trixie -t paperclip-static-site:size-trixie .
```

Compare:

```
docker images paperclip-static-site --format "table {{.Repository}} {{.Tag}}
{{.Size}}"
docker image inspect paperclip-static-site:size-default --format "default
bytes={{.Size}}"
docker image inspect paperclip-static-site:size-alpine --format "alpine bytes=
{{.Size}}"
docker image inspect paperclip-static-site:size-trixie --format "trixie bytes=
{{.Size}}"
```

기록 표:

Variant	Base image	Local image size	해석
default	nginx:stable	학생 기록	일반 계열 기준
alpine	nginx:stable-alpine	학생 기록	보통 가장 작음
trixie	nginx:stable-trixie	학생 기록	Debian trixie 기반, 패키지 호환성/운영 표준 확인

판단 기준

증거	해석
CACHED	이전 layer 재사용
source COPY 이후 rebuild	앱 파일 변경 영향
image size 증가	base image, context, dependency, build output 포함 여부 점검
alpine/default/trixie 차이	base image 선택이 size와 운영 특성에 직접 영향
tag size-*	같은 앱을 base image별로 비교하기 위한 reference

학생이 말할 수 있어야 하는 문장

Image size는 단순 디스크 문제가 아니라 build, push, pull, deploy, registry storage, network 비용에 영향을 준다.

Base image를 바꾸면 앱 코드는 같아도 결과 image size가 달라진다.

Alpine은 작지만 항상 정답은 아니며, 필요한 패키지와 조직 표준을 함께 봐야 한다.

.dockerignore는 build context를 줄이고 secret과 불필요한 결과물이 image에 들어가는 것을 막는다.

핵심 포인트

layer/cache를 설명하지 못하면 Docker build가 왜 빠르거나 느린지 말할 수 없다. image size를 설명하지 못하면 왜 base image와 .dockerignore를 신경 써야 하는지도 말할 수 없다.

다음 연결

다음 교시는 정상 경로를 일부러 깨서 build/run/verify 실패를 분류한다.